



PROJECT OVERVIEW

DairyOS

An AI-powered, multi-tenant dairy farm management SaaS — from farm operations and herd tracking to enterprise-grade authentication and platform administration.

Prepared as a full project summary

Stack: Express · PostgreSQL · React · Vite · TypeScript

Document generated 2026-07-25

1. What DairyOS Is

DairyOS is a full-stack SaaS platform for running a dairy farm: tracking every cow, barn, milk record, and employee, while an AI layer surfaces daily priorities, health risks, and financial insight. It is built multi-tenant from the ground up — a single account can own or belong to several independent farms, each with its own herd, team, and data, fully isolated from every other tenant on the platform.

Express + PostgreSQL (raw SQL)

React 18 + Vite

TypeScript end-to-end

JWT + refresh-token sessions

npm workspaces monorepo

apps/server

Express API, ~40 route modules, raw parameterized SQL via a shared query helper, zod validation on every input boundary.

apps/web

React + Vite SPA, hash-based routing, Context-driven auth/theme/plan state, Chart.js visualizations.

database

22 sequential migrations plus a demo seed — roles, permissions, farms, cows, security tables, and more.

docs

OpenAPI spec describing the public API surface.

2. Core Farm Management

The foundation: herd records, barns, milk and feed logs, health and breeding history, employee HR (attendance, shifts, payroll, leave, training), finance, customers, and a photo gallery — each farm-scoped and permission-gated.

Interactive Farm Map

A zoomable, pannable 2D digital twin of the farm. Every cow appears as a live marker colored by status (healthy / attention / sick / pregnant / breeding). Clicking a barn shows real occupancy, milk today, average yield, health counts, and feed remaining.

Smart Heat Maps

Health, milk production, and feed consumption rendered as zone-level color intensity.

Weather Overlay

Temperature, humidity, wind, rain, and a real Temperature-Humidity Index (THI) heat-stress calculation, with AI recommendations (e.g. open ventilation, adjust feeding times).

AI Zone Recommendations

Rule-engine suggestions driven by live zone data — not just numbers, but "do this next."

Timeline Slider

Yesterday / Today / Last Week / Last Month / Forecast — reconstructs how the farm looked at any point, deterministically (no fabricated data).

Calving Map

Highlights expected calving locations, recently calved cows, and high-risk pregnancies based on real breeding history.

3. Authentication & Onboarding

A production-shaped auth system supporting one email account managing multiple farms — the flow a real SaaS needs: Landing → Get Started → Choose Account Type → Create Account → Verify Email → Create Farm → Farm Setup Wizard → Invite Employees → Dashboard .

Sign-up: 8 account types, each with real consequences

ACCOUNT TYPE	WHAT HAPPENS
Farm Owner / Dairy Cooperative / Research Institution	Owner flow — proceeds to create a farm
Farm Manager / Veterinarian / Farm Worker / Accountant	Team-member flow — waits for an invite, with a role hint shown
Student / Demo User	Auto-provisions a fully populated sandbox farm (14 cows, milk history, breeding records) in seconds

Farm Setup Wizard

A four-step guided flow after farm creation: upload a logo/photo, create barns (preset suggestions or custom), import cows (CSV upload, Excel upload, or manual entry, all feeding one bulk-import endpoint), then invite the team.

Dashboard "Farm Setup" checklist

Instead of an empty dashboard, a new farm sees a real, computed completion checklist (Account, Email, Farm, Barns, Employees, Add Cows, Configure Feeding, Connect AI Advisor) with a live percentage — every item is a genuine database check, not a stored flag, and the card disappears once setup reaches 100%.

4. Security Features

Email + password

bcrypt-style hashing via Postgres `crypt()`, password-strength meter on every password field, show/hide toggle.

Phone number + OTP

A full alternate login method — request a code, verify, session issued.

Google / Microsoft / Apple login

Scaffolded end-to-end (routes, buttons, account linking) but deliberately left disabled until real OAuth credentials are supplied — no fabricated integration.

Two-Factor Authentication

Real RFC 6238 TOTP, hand-rolled (no dependency), verified interoperable with standard authenticator apps.

CAPTCHA

Self-hosted arithmetic challenge after repeated failed logins — a real stand-in until a hosted reCAPTCHA/hCaptcha key exists.

Account logout

Locks for 15 minutes after 5 failed attempts — enforced even against the correct password while locked.

Login history & device management

Every login attempt logged; active sessions listed with device/IP, individually revocable.

Email alerts for new logins

Fires only the first time a given device signs in successfully — not on every login.

5. Roles & Permissions

Eight roles, each mapped to real, enforced permissions — not cosmetic labels:

ROLE	SCOPE
Super Admin	Platform-wide — every farm, every account (see Section 6)
Farm Owner	Full control of their own farm(s)
Farm Manager	Daily operations — herd, milk, health, tasks
Veterinarian	Health records only
Worker	Daily task and milk updates
Accountant	Finance only
Milk Collector	Milk records only
Viewer	Read-only access

A "Members" tab on the Team page lets an owner or manager invite teammates by role and see who has joined or is still pending — the capability the onboarding flow always promised but, until this pass, didn't actually expose after the first-run wizard.

6. Platform Administration

A genuine platform-wide Super Admin layer, architecturally separate from every farm-scoped role: a flag on the account, not a row in the roles table, because it must span every tenant rather than being scoped to one farm at a time. The Platform Admin console (visible only to Super Admin accounts) gives real cross-tenant visibility — every farm, every user, aggregate counts, unverified accounts — backed by dedicated, Super-Admin-only API routes.

Security hardening performed on this codebase: while building the Super Admin layer, an audit found that the ordinary, farm-scoped `administrator` role was being used across roughly ten backend route files as a bypass for farm-ownership checks. Because *every* farm owner holds that role, any Farm Owner could — before this fix — read or write another tenant's cows, health records, farm members, and even see every other farm in their own farm switcher, simply by knowing or guessing an ID. This was closed by introducing a true, separately-tracked `isSuperAdmin` check and applying it consistently everywhere the old shortcut existed, then verifying in both directions with live two-tenant tests: ordinary owners are now fully isolated to their own data, while genuine Super Admin accounts retain full cross-farm access.

7. Running the Project

COMPONENT	URL / COMMAND
API server	<code>http://localhost:4000</code> — <code>npm run dev</code> in <code>apps/server</code>
Web app	<code>http://localhost:5173</code> — <code>npm run dev</code> in <code>apps/web</code>
Demo login	<code>admin@greenfield.test</code> / <code>ChangeMe123!</code>
Database	PostgreSQL, migrations in <code>database/migrations</code> , demo data in <code>database/seeds</code>

Both services are typechecked (`tsc`) and build cleanly (`vite build`) as of this document. Every feature described above has been verified against a running instance — real database state, real HTTP requests, and browser screenshots — rather than assumed from the code alone.